

GOVERNMENT HYDRO ENGINEERING COLLEGE

BANDLA, BILASPUR(H.P)

G-Hydro HLM: A web based Hostel Leave Management System for GHEC

PROJECT REPORT - CAPSTONE PROJECT

(Code : CSEE-611P)

Submitted to the

Department of Computer Science and Engineering

(Artificial Intelligence and Data Science)

Submitted by :

Student Name	University Roll no.
Priyal Sood	23010503043
Ritisha dass	23010503049
Sakshi	23010503050
Sourav	23010503065
Varnika Samkaria	23010503073

Under the Supervision of

Dr. Shashi Gurung

Associate Professor CSE

(Government Hydro Engineering College Bandla)



Government Hydro Engineering College

Bandla, Bilaspur (Himachal Pradesh)

(January- May),2026

Declaration

We hereby declare that the Capstone Project entitled “**G-Hydro HLM: A web based Hostel Leave Management System for GHEC**” submitted by us for the Degree of Bachelor of Engineering in Computer Science and Engineering is the result of our original and independent research work carried out under the guidance of Supervisor Dr. Shashi Gurung and it has not been submitted to any other university or institute for the award of any degree or diploma. We also hereby declare that this written submission represents my own ideas in my own words. We also declare that the other ideas, which are included in project, have adequately provided the reference from the original reported work.

We have taken care in all respect to honor the intellectual Property Right and have acknowledged the contribution of others for using them in this project work for academic purpose. We further declare that in case of any violation of intellectual Property Right or Copy Right, We as the candidate will be fully responsible for the same. My Supervisor, Head of the Department and Institute Head should not be held responsible for any violation of copy right if found at any stage of my degree.

Place Bandla

Date May, 2026

Signature of the Students

Certificate

This is to certify that the Capstone Project entitled “**G-Hydro HLM: A web based Hostel Leave Management System for GHEC**” submitted by

Priyal Sood(23010503043)

Ritisha Dass(23010503049)

Sakshi(23010503050)

Sourav Kumar(23010503065)

Varnika Samkaria(23010503073)

for the award of the degree of the Bachelor of Engineering in Computer Science and Engineering, Government Hydro Engineering College Bandla, Bilaspur, Himachal Pradesh, India is entirely based on the work carried out by him under our supervision and guidance, The work recorded, embodies the original work of the candidate and has not been submitted for the award of any degree, or diploma of any university and institute, according to the best of our knowledge.

Date May, 2026

Place GHEC Bandla

Signature of Supervisor

Acknowledgement

I would like to express my sincere gratitude to the following people who supported and guided me during the development of this project:

1. Project Guide and Faculty Members I am deeply thankful to my project guide and faculty members for their valuable guidance, suggestions, and continuous support throughout the development of this project.
2. College Department I would also like to thank the Department of Computer science and Engineering for providing the necessary resources, environment, and opportunity to complete this project successfully.
3. Friends and Classmates I am grateful to my friends and classmates for their helpful discussions, suggestions, and encouragement during the project work.
4. Family Members Finally, I would like to thank my family for their constant support, motivation, and encouragement throughout my academic journey .

Index

Section	Title	Page No.
I.	Title Sheet	1
II.	Certificate	2
III.	Acknowledgement	3
IV.	Index	5
V.	Abstract	5
CHAPTER 1	INTRODUCTION	9
1.1	Title of the Project	9
1.2	Problem Statement	9
1.3	Objective of the System	9
CHAPTER 2	LITERATURE SURVEY	11
2.1	Analysis of Existing Manual Systems	11
2.1.1	The Paper-Based Bottleneck and Data Integrity Risks	11
2.1.2	Administrative Overhead and Operational Inefficiency	11
2.2	Comparative Study of Digital Solutions	12
2.2.1	Evolution of Web-Based Automation	12
2.2.2	IoT and Hardware Integration for Security	12
2.2.3	Data Privacy and Role-Based Access	13
CHAPTER 3	SYSTEM ANALYSIS	14
3.1	Proposed System	14
3.2	Feasibility Study	14
3.3	Hardware & Software Requirements	14

Section	Title	Page No.
3.3.1	Software Requirements	14
3.3.2	Hardware Requirements	15
3.4	Functional & Non-Functional Requirements	15
3.4.1	Functional Requirements	15
3.4.2	Non-Functional Requirements	15
CHAPTER 4	SYSTEM DESIGN & ARCHITECTURE	16
4.1	Data Flow Diagram (DFD)	16
4.2	Database Architecture (ER Diagram)	16
4.2.1	The ACID Compliance Framework	16
4.2.2	Structural Advantages of RDBMS vs. Manual Registers	17
4.3	System Process Flowchart	18
4.3.1	Workflow Execution: Life of a Leave Request in G-Hydro HLM	19
CHAPTER 5	PLAN OF WORK	21
5.1	Project Timeline (Gantt Chart)	21
5.1.1	Phase I: Requirement Gathering & Analysis	21
5.1.2	Phase II: System Design & Architecture	21
5.1.3	Phase III: Core Development & Coding	21
5.1.4	Phase IV: Testing & Quality Assurance	22
5.1.5	Phase V: Final Documentation & Deployment	22
CHAPTER 6	SECURITY & RELIABILITY FRAMEWORK	23
6.1	Cryptographic Credential Protection (Bcrypt)	23

Section	Title	Page No.
6.2	Transport Layer Security (SSL/TLS)	23
6.3	Role-Based Access Control (RBAC)	23
6.4	Operational Reliability & Fail-Safe Redundancy	24
CHAPTER 7	IMPLEMENTATION	25
7.1	Tools & Technologies	25
7.2	System Interface Gallery (Dashboards)	25
7.3	Automated Rule Engine Logic	28
CHAPTER 8	TESTING & VALIDATION	30
8.1	Unit & Integration Testing	29
8.2	Test Case Table	29
8.3	Fail-Safe & Stress Testing	30
CHAPTER 9	RESULTS AND DISCUSSION	31
9.1	Performance Metrics Analysis	31
9.2	Executive Monitoring Impact	31
CHAPTER 10	CONCLUSION & FUTURE SCOPE	33
10.1	Conclusion	33
10.2	Future Scope	33
CHAPTER 11	REFERENCES	34
11.1	Technical Documentation	34
11.2	Books & Academic Journals	34

Section	Title	Page No.
11.3	Web Resources & Standards	34
CHAPTER 12	ANNEXURES (SOURCE CODE)	35
12.1	Backend Database Engine (db.py)	35
12.2	Data Integrity & Synchronization (fix_db.py)	36
12.3	System Patching & Maintenance (patch.py)	37
12.4	Frontend Template: Director Dashboard (director_dashboard.html)	37

CHAPTER 1 : INTRODUCTION

11.3 Title of the Project:

G-Hydro HLM: A web based Hostel Leave Management System for GHEC

Introduction:

In many hostels, the leave process is still done manually using paper forms and registers. This method takes more time and can cause mistakes, loss of records, and confusion for both students and hostel authorities. Students have to fill out forms, and wardens need to check and approve them manually. The G-Hydro HLM is a digital system that helps to manage hostel leave in an easy and organized way. Through this system, students can apply for leave online, and wardens can approve or reject the request. All leave records are stored safely in one place, which makes tracking and managing student leaves simple and efficient. The system provides a centralized platform that allows authorized users to manage hostel leave requests easily and efficiently.

1.2 Problem Statement :

Manual hostel leave management suffers from several issues such as:

- Hostel leave management is mostly done using manual paper-based systems.
- Students must physically visit multiple authorities to apply for leave.
- The manual process is time-consuming and inefficient.
- Chances of data manipulation or loss
- Difficulty in tracking student movement
- No centralized database for leave history
- Maintaining separate registers for leave, mess-off, and permanent leave increases workload.

1.3 Objective of the System :

The main objective of G-Hydro HLM are :

- To digitize the hostel leave application process
- To reduce manual work and errors
- To provide leave approval and status updates
- To maintain a centralized database of student leave records
- To improve hostel security and administration efficiency

- To maintain accurate and organized leave records for hostel administration
- To save time and effort for both students and hostel staff.

Tools and Technologies :

Front end : HTML5, CSS3, Bootstrap

Back end : Python, Flask, Flask-SQLAlchemy, Flask-WTF, AI- ML Integration

Database : SQLite (with SQLAlchemy)

Development tools : Visual Studio Code (VS Code), Pipenv / Virtualenv, Postman

CHAPTER 2: LITERATURE SURVEY

2.1 Analysis of Existing Manual Systems

The current administrative framework at Government Hydro Engineering College (GHEC) relies on traditional, paper-based logging mechanisms. While these systems have been the standard for years, an in-depth analysis reveals significant systemic vulnerabilities that hinder institutional efficiency and student safety.

2.1.1 The Paper-Based Bottleneck and Data Integrity Risks

Manual registers serve as the primary “database” for student movement, but they present several critical failures in a modern academic environment:

- **Data Redundancy and Fragmentation:** Information is often duplicated across multiple registers (e.g., separate logs for the hostel gate, the warden’s office, and the mess department). This fragmentation makes it nearly impossible to maintain a “Single Source of Truth,” leading to conflicting records regarding a student’s actual location.
- **Risk of Physical Record Loss:** Paper documents are susceptible to environmental damage, such as moisture, fire, or simple wear and tear. Once a register page is damaged or lost, the historical leave data for hundreds of students is permanently erased, creating a significant gap in institutional archives.
- **Unauthorized Data Manipulation:** Manual logs lack a “Read-Only” or “Audit Trail” feature. It is relatively easy for unauthorized individuals to alter entries, forge signatures, or backdate leave requests without leaving a trace of the modification.

2.1.2 Administrative Overhead and Operational Inefficiency

The “human cost” of manual management results in a slow, cumbersome experience for both the staff and the students:

- **The Physical Transit Requirement:** Students are currently required to physically navigate the campus to collect signatures from the Caretaker, Superintendent, and Warden. This “walk-in” requirement is not only time-consuming but often leads to delays if an official is in a meeting or away from their desk, sometimes stretching a simple leave process into a multi-day task.
- **Lack of Real-Time Movement Tracking:** From a security standpoint, the current system is “reactive” rather than “proactive.” Hostel authorities cannot instantly see how many students are currently “OUT” or “IN” without manually counting lines

in a register. This lack of visibility is a major safety concern during emergencies or campus audits.

- **Mess Resource Mismanagement:** Because leave data is not instantly synced with the mess department, there is a recurring lag in implementing “Mess-Off” rebates. This leads to food wastage and financial inaccuracies that a digital, synchronized system like G-Hydro HLM is designed to eliminate.

2.2 Comparative Study of Digital Solutions

A comparative analysis of existing digital frameworks was conducted to identify the most effective methodologies for hostel management. By evaluating established research, **G-Hydro HLM** was engineered to integrate high-performance backend logic with advanced security protocols.

2.2.1 Evolution of Web-Based Automation

The transition from desktop applications to web-based environments has been a critical focus of recent research.

- **Kumar et al. (2020):** Demonstrated that “Online Hostel Management Systems” utilizing modern web frameworks drastically reduce the “administrative lead time”—the gap between a student’s request and an official’s action. Their findings suggest that centralized servers provide a synchronous environment where data is updated instantly across all departments.
- **Rani & Devi (2021):** Explored the efficacy of PHP and Python in handling large-scale student databases. Their comparative study concluded that Python-based frameworks like **Flask** offer superior readability and modularity, making them ideal for academic institutions requiring custom logic, such as the specific mess-off rules used at GHEC.

2.2.2 IoT and Hardware Integration for Security

Modern hostel security has evolved beyond manual identity checks to include automated hardware-software synchronization.

- **Patel et al. (2022):** Investigated “IoT-based Hostel Management,” which emphasizes the importance of a “Digital Gatekeeper”. Their research justifies the use of **QR-code verification** as a high-speed, contactless identification method.
- **Application in G-Hydro HLM:** Following Patel’s methodology, our system utilizes dynamic QR codes to log timestamps with millisecond precision. This

eliminates “proxy attendance” and ensures that the student’s presence or absence in the database is backed by a physical hardware scan.

2.2.3 Data Privacy and Role-Based Access Control (RBAC)

Protecting student data from unauthorized internal and external access is a primary architectural requirement.

- **Gupta & Verma (2021):** Established the “Principle of Dashboard Isolation,” which argues that administrative power should be tiered to prevent “vertical movement”—where a lower-level official gains access to executive data.
- **The Director’s Analytics Suite:** Built upon the principles of Gupta & Verma, G-Hydro HLM implements **Read-Only Administrative Isolation**. This provides the Director with complete institutional oversight through data visualization while programmatically barring the account from altering student records or approving leaves. Furthermore, the inclusion of **Bcrypt cryptographic hashing** ensures that even at the database level, student credentials remain computationally protected.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Proposed System

The G-Hydro HLM (Hostel Leave Management System) is designed to replace inefficient paper-based workflows with a secure, multi-role digital ecosystem.

- **Centralized Architecture:** Unlike manual registers, the proposed system uses a centralized SQLite database to ensure a single, synchronized record of student movement.
- **Automated Workflow:** It introduces a digital approval hierarchy where requests are routed automatically from students to superintendents and wardens, reducing physical transit time.
- **Verification Protocols:** The system implements primary (QR-code) and secondary (Manual UID) verification protocols to maintain 100% operational uptime at the hostel gate.

3.2 Feasibility Study

A feasibility analysis was conducted to ensure the project is viable within the college environment.

- **Technical Feasibility:** The use of Python and Flask is highly feasible as they are robust, scalable, and compatible with the existing GHEC infrastructure.
- **Economic Feasibility:** The system utilizes open-source technologies (SQLite, Flask, Bootstrap), eliminating licensing costs and making it a cost-effective solution for the institution.
- **Operational Feasibility:** With intuitive dashboards, the system requires minimal technical training for hostel staff, ensuring smooth adoption.

3.3 Hardware & Software Requirements

To maintain the “Secure Digital Framework,” the following specifications are required:

3.3.1 Software Requirements

- **Operating System:** Windows 10/11 or Linux (Ubuntu) for the host server.
- **Development Environment:** Visual Studio Code (VS Code).
- **Backend Framework:** Python 3.x with Flask and SQLAlchemy.
- **Frontend Stack:** HTML5, CSS3, and Bootstrap for responsive design.

- **Database:** SQLite3 for ACID-compliant data persistence.

3.3.2 Hardware Requirements

- **Server/Host:** Minimum 4GB RAM and Intel i3 Processor (or equivalent).
- **Client Devices:** Smartphones or Tablets with camera functionality for QR scanning.
- **Network:** Campus Wi-Fi or local area network (LAN) for server-client communication.

3.4 Functional & Non-Functional Requirements

3.4.1 Functional Requirements

These define the core actions the system must perform:

- **Registration/Login:** Secure access for students and staff using encrypted credentials.
- **Leave Application:** Interface for students to input destination, reason, and dates.
- **Multi-tier Approval:** Automated routing of requests for superintendent and warden authorization.
- **Executive Monitoring:** Real-time data visualization of occupancy and leave logs for the Director.

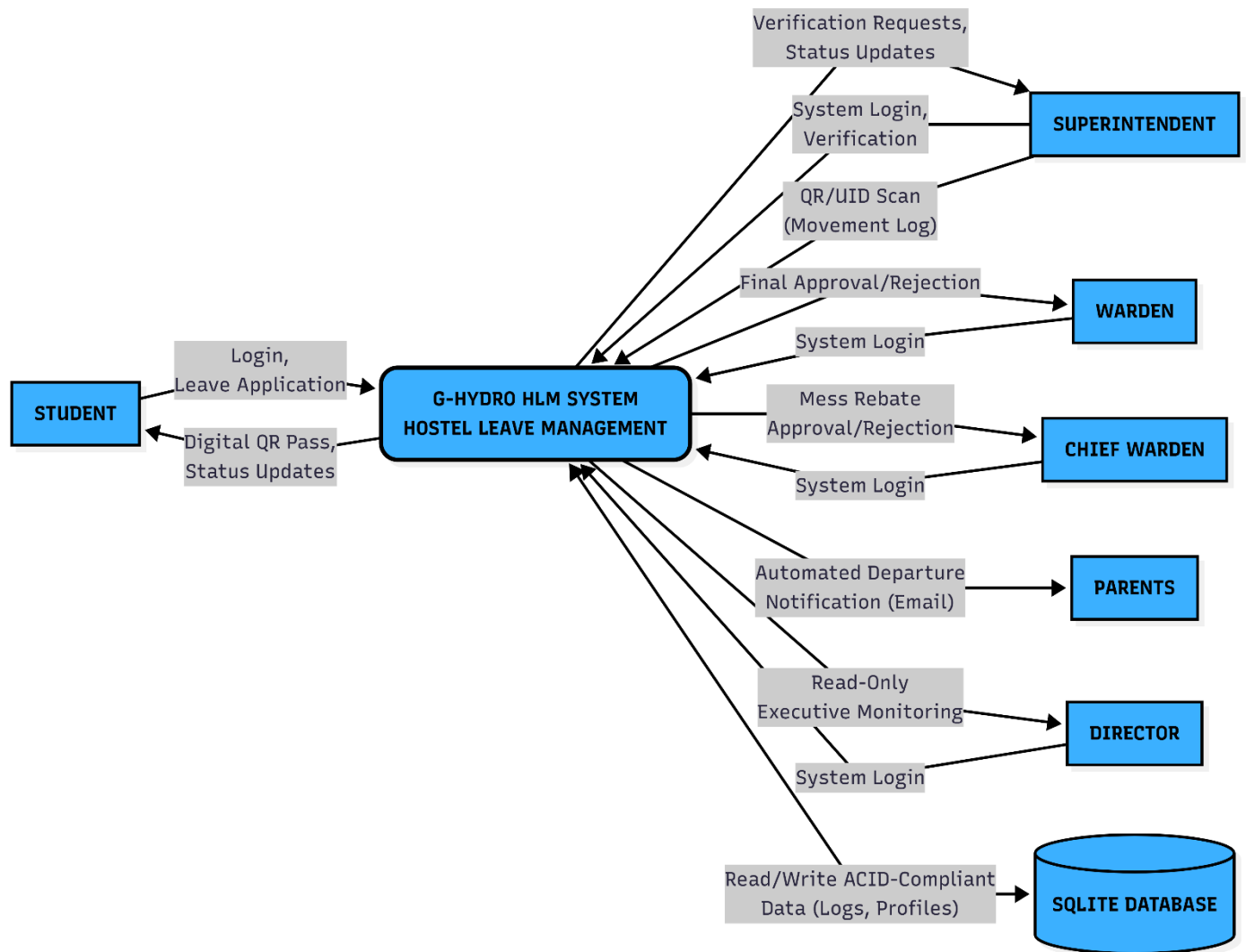
3.4.2 Non-Functional Requirements

These define the quality attributes of the system:

- **Security:** Implementation of Bcrypt hashing and SSL/TLS for data protection.
- **Reliability:** High system availability through dual-verification (QR and UID).
- **Scalability:** Database architecture capable of handling the entire GHEC hostel population.
- **Usability:** Clean, role-specific interfaces that work across both mobile and desktop views.

CHAPTER 4: SYSTEM DESIGN & ARCHITECTURE

4.1 Data Flow Diagram (DFD)



The DFD represents the logical flow of information through the G-Hydro HLM ecosystem. It illustrates how student data moves from the portal, through the automated validation logic, to the administrative dashboards for approval.

4.2 Database Architecture

The G-Hydro HLM ecosystem utilizes a centralized Relational Database Management System (RDBMS) implemented via the SQLite engine. This ensures high-speed retrieval and data integrity across the institution.

4.2.1 The ACID Compliance Framework

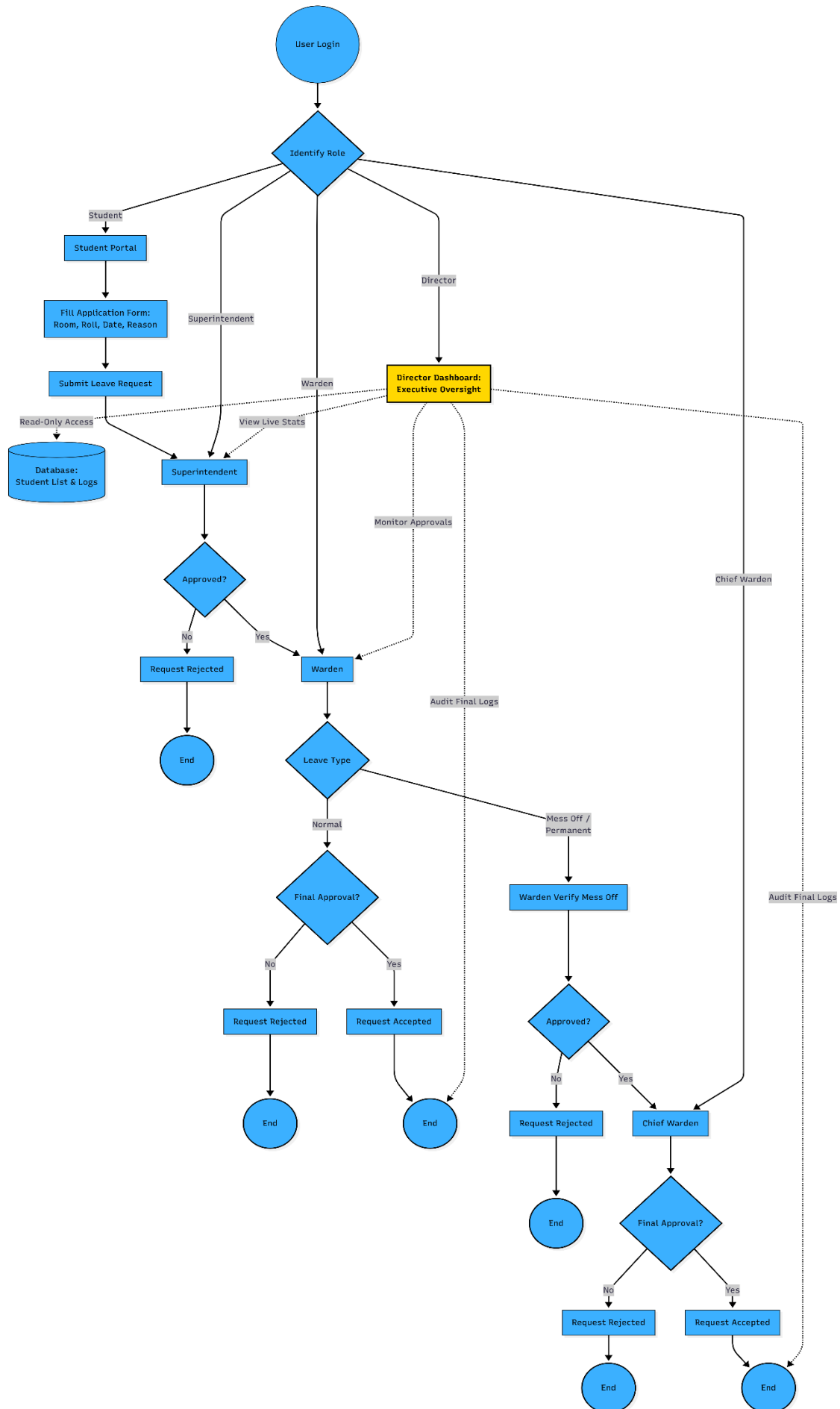
To ensure the system remains robust during concurrent administrative access, the data layer adheres to ACID principles:

- **Atomicity:** Ensures every leave request is treated as a single “unit” that either succeeds or fails completely.
- **Consistency:** Guarantees that student records and security credentials remain synchronized.
- **Isolation:** Ensures simultaneous actions (e.g., Warden approval vs. Director viewing analytics) do not interfere with each other.
- **Durability:** Once a status is toggled to “OUT” or “IN,” the record is permanently saved against data corruption.

4.2.2 Structural Advantages of RDBMS vs. Manual Registers

Feature	Manual Paper Registers	G-Hydro HLM (SQLite)
Data Integrity	High risk of error/loss	ACID-Compliant automation
Search Speed	Minutes (physical searching)	Milliseconds (SQL Query)
Concurrency	One person at a time	Multi-role access enabled
History	Difficult to track old logs	Centralized leave history

4.3 System Process Flowchart



4.3.1 Workflow Execution: Life of a Leave Request in G-Hydro HLM :

The workflow of a leave request within the G-HydroHLM ecosystem is designed as a multi-tier validation process to ensure security, administrative oversight, and student safety. It begins when a student submits a digital application through their portal, providing the destination, reason, and duration.

1. **Automated Validation:** Before the request reaches any official, the system's backend logic automatically checks the duration against pre-defined hostel policies, such as the 5-day and 10-day mess-off thresholds.
2. **Multi-Level Approval:** Once submitted, the request is routed to the Caretaker's dashboard for preliminary verification. Following the Caretaker's initial endorsement, the application escalates to the Warden, and in cases of extended leave, to the Chief Warden for final authorization.
3. **Automated Validation:** Before the request reaches any official, the system's backend logic automatically checks the duration against pre-defined hostel policies, such as the 5-day and 10-day mess-off thresholds.
4. **Multi-Level Approval:** Once submitted, the request is routed to the Caretaker's dashboard for preliminary verification. Following the Caretaker's initial endorsement, the application escalates to the Warden, and in cases of extended leave, to the Chief Warden for final authorization.
5. **Digital Pass Generation:** Upon receiving formal authorization, the student's status is updated to "Approved," and a unique digital gate pass is activated within their account.
6. **QR-Verified Movement (Primary Protocol):** The process culminates at the hostel gate, where the student presents their unique QR Code. The Caretaker scans this code using the system's integrated scanner, which instantly logs the exact timestamp and toggles the student's database status from "In" to "Out." Executive Monitoring (The Director's View): Throughout this entire process, the Director has access to a centralized, read-only dashboard. The Director can view real-time statistics, pending request counts, and student movement logs to ensure administrative transparency without interfering in the approval chain.
7. **Emergency Protocol and Manual Override (Secondary Protocol):** To ensure functionality during emergencies or hardware failures (such as phone battery depletion), the system incorporates a Manual Override feature. Each student is assigned a persistent Unique Identification Number (UID). In critical situations, the student can report directly to the Caretaker, who validates their identity via the UID

and manually updates their status. This dual-layer approach ensures that while the system is digitally driven, it remains adaptable to real-world hostel scenarios where student safety is the priority.

CHAPTER 5: PLAN OF WORK

5.1 Project Timeline (Gantt Chart)

The development of the G-Hydro HLM ecosystem was executed over a period of 18 weeks. To ensure the delivery of a “Secure Digital Framework,” the project followed a structured timeline categorized into five sequential phases. This systematic approach ensured that the transition from a manual paper-based register to an automated RDBMS-driven system remained organized and technically sound.

5.1.1 Phase I: Requirement Gathering & Analysis

The initial phase of the project focused on defining the operational boundaries and institutional needs of Government Hydro Engineering College (GHEC).

- **Problem Identification:** Conducted a thorough audit of the existing manual leave registers to identify risks of data loss, proxy entries, and slow retrieval times.
- **Stakeholder Interviews:** Consulted with hostel superintendents and wardens to understand the approval hierarchy and the specific need for a multi-tier authorization workflow.
- **Defining Constraints:** Established the core technical requirements, such as the 5-day and 10-day mess-off threshold logic and the necessity of a secondary manual override protocol for hardware failures.

5.1.2 Phase II: System Design & Architecture

In this phase, the project was translated from abstract requirements into a technical blueprint.

- **Diagrammatic Modeling:** Designed the Consolidated Level-0 Data Flow Diagram (DFD) and Entity-Relationship Diagram (ERD) to map data movement and storage.
- **Unified Role Design:** Developed the architecture for the Unified Superintendent Dashboard, merging administrative endorsement duties with physical gate-security tasks (QR scanning).
- **Database Schema:** Structured the SQLite database to ensure ACID compliance, focusing on table relationships between student profiles, leave logs, and secure Bcrypt-hashed credentials.

5.1.3 Phase III: Core Development & Coding

This phase involved the primary implementation of the software using the Python/Flask stack.

- **Backend Engineering:** Coded the central routing logic, session management, and server-side validation using the Flask framework.
- **Security Integration:** Implemented cryptographic layers, specifically the Bcrypt algorithm for password salting and hashing, to protect user data from unauthorized access.
- **Feature Development:** Developed the dynamic QR code generation engine and the real-time Director's Analytics Dashboard to provide executive oversight without interfering in daily hostel operations.

5.1.4 Phase IV: Testing & Quality Assurance

To ensure the system was “Real-World Ready,” it underwent rigorous testing in a controlled environment.

- **Unit & Logic Testing:** Verified individual components, such as checking if the system correctly flags “Mess-Off” status based on leave duration.
- **Role-Based Access Control (RBAC) Testing:** Confirmed that different roles (Student, Superintendent, Warden, Director) could only access their assigned dashboards and functions.
- **Stress & Redundancy Testing:** Simulated hardware failures to verify the reliability of the Manual UID Override feature, ensuring that student movement can still be logged if a mobile device is unavailable.

5.1.5 Phase V: Final Documentation & Deployment

The final phase focused on finalizing the project for academic submission and institutional handover.

- **Deployment Scripting:** Developed an “Automated System Initializer” to allow non-technical staff to launch the G-Hydro HLM server with a single action.
- **Report Compilation:** Finalized the 60–70 page technical report, detailing the literature survey, design methodology, and future scope.
- **Performance Review:** Conducted a final pilot run to ensure the system met all institutional security standards and was ready for the final presentation before the external examiner.

CHAPTER 6: SECURITY & RELIABILITY FRAMEWORK

The integrity of the G-Hydro HLM ecosystem relies on the uncompromising security of student data and the absolute reliability of movement records. The system implements a multi-layered security architecture designed to protect against unauthorized data manipulation, network-level interception, and administrative overreach.

6.1 Cryptographic Credential Protection (Bcrypt)

User privacy is enforced through an advanced cryptographic hashing layer. The system architecture strictly prohibits the storage of plain-text passwords in the SQLite database. Instead, it utilizes the Bcrypt algorithm, which is a password-hashing function based on the Blowfish cipher.

- **Mechanism:** During registration or profile updates, the system takes the user's password and passes it through the Bcrypt function. This process automatically incorporates a unique "Salt"—a random string added to the password—for every record.
- **Engineering Benefit:** This ensures that even in the event of unauthorized access to the database layer, the original credentials remain computationally impossible to reverse-engineer. Bcrypt's adaptive nature also allows the system to increase the "work factor" (computational cost) over time to protect against future brute-force attacks.

6.2 Transport Layer Security (SSL/TLS)

To secure communications across the GHEC campus network and via remote access tunnels, the system implements Transport Layer Security (TLS), commonly referred to as SSL. This ensures that all data moving between the student's smartphone or the Superintendent's laptop and the central server is encrypted.

- **Implementation:** By utilizing X.509 digital certificates, a secure HTTPS tunnel is established. The server uses a private key to sign data packets, while the client uses a public key to verify them.
- **Protection:** This architecture mitigates "Man-in-the-Middle" (MITM) attacks. It ensures that sensitive data—such as student UIDs, parental contact details, and specific leave reasons—cannot be intercepted by unauthorized parties monitoring the college Wi-Fi or local area network.

6.3 Role-Based Access Control (RBAC) The system enforces a strict hierarchical permission model to prevent horizontal or vertical privilege escalation. Every user session is

verified against a “Role” attribute stored in the database, ensuring that users can only access features relevant to their specific duties.

- **Student Role:** Permissions are strictly limited to personal profile management, submission of leave applications, and viewing personal movement history.
- **Superintendent Role (Unified Authority):** This role is granted access to the Master Dashboard. It includes permissions for preliminary request endorsement, real-time QR scanning at the gate, and the Manual UID Override protocol.
- **Warden / Chief Warden Role:** These roles possess the final authorization rights. They can approve or reject requests that have been endorsed by the Superintendent but are programmatically restricted from performing gate-scans.
- **Director Role (Executive Oversight):** The system implements Read-Only Administrative Isolation for this role. The Director can monitor all hostel statistics and audit logs but is barred from altering operational records or approving individual leaves. This ensures total institutional transparency without infringing upon the duties of the wardens.

6.4 Operational Reliability & Fail-Safe Redundancy

Reliability in the **G-Hydro HLM** ecosystem is achieved through a dual-layer verification architecture. The framework is engineered to maintain 100% operational uptime, ensuring that hostel security is never compromised by hardware or technology failures.

- **Dynamic QR vs. Persistent UID:** While the primary tracking method utilizes a dynamic, time-sensitive QR code for high-speed automated logging, every student is also assigned a persistent **Unique Identifier (UID)**. This UID serves as a static data anchor within the SQLite database.
- **Manual Override Protocol:** In emergency scenarios—such as a student’s mobile device being unavailable or a failure in the scanning hardware—the **Superintendent** can utilize the Manual Override interface. By entering the student’s UID, the Superintendent can verify the individual’s identity and manually update their movement status.
- **Audit Logging & Accountability:** To prevent administrative overreach, every manual override is cryptographically logged with a unique timestamp and the ID of the authorizing official. These entries are immediately flagged and made visible on the **Director’s Dashboard**, ensuring that manual actions are subject to the same level of executive audit as automated scans.

CHAPTER 7: IMPLEMENTATION

The implementation phase of the G-Hydro HLM project involved translating the conceptual designs and security frameworks into a functional, modular software ecosystem. The system was developed with a focus on scalability, ensuring that the architecture could handle the high volume of student traffic at Government Hydro Engineering College (GHEC).

7.1 Tools & Technologies

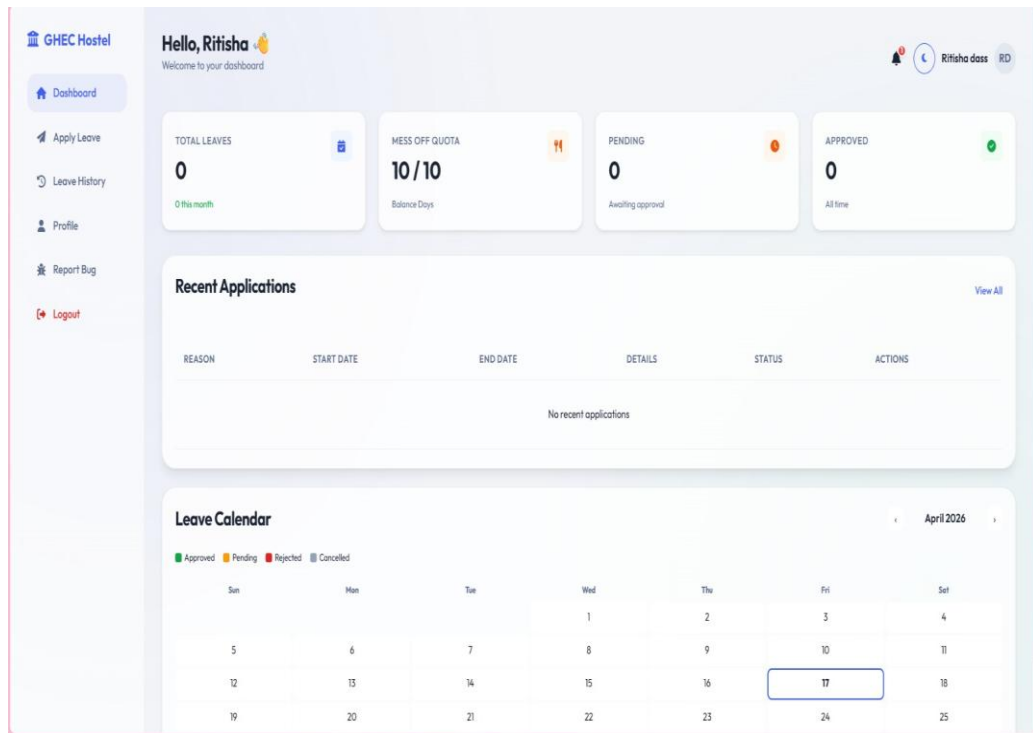
The selection of the “Tech Stack” was based on the need for high-speed processing, ACID-compliant data persistence, and a responsive user interface that functions across both mobile and desktop platforms.

- **Backend Framework (Python & Flask):** The core logic is powered by Python, chosen for its robust libraries and efficiency in data handling. The Flask framework was utilized to manage server-side routing, secure session handling, and the integration of the Bcrypt security layer.
- **Database Engine (SQLite3):** For data persistence, SQLite was implemented. This relational database ensures that every leave request and movement log is stored with high integrity, supporting complex SQL queries for real-time analytics.
- **Frontend Technologies (HTML5, CSS3, Bootstrap):** The user interface was built using a mobile-first approach. Bootstrap was integrated to ensure that the Superintendent can scan QR codes easily on a handheld device, while the Director can view large-scale analytics on a desktop.
- **Development & Testing Hardware:** The system was developed and hosted on a Dell Chromebook environment and field-tested using a Redmi 14C 5G smartphone to verify the efficiency of the integrated QR scanner hardware.

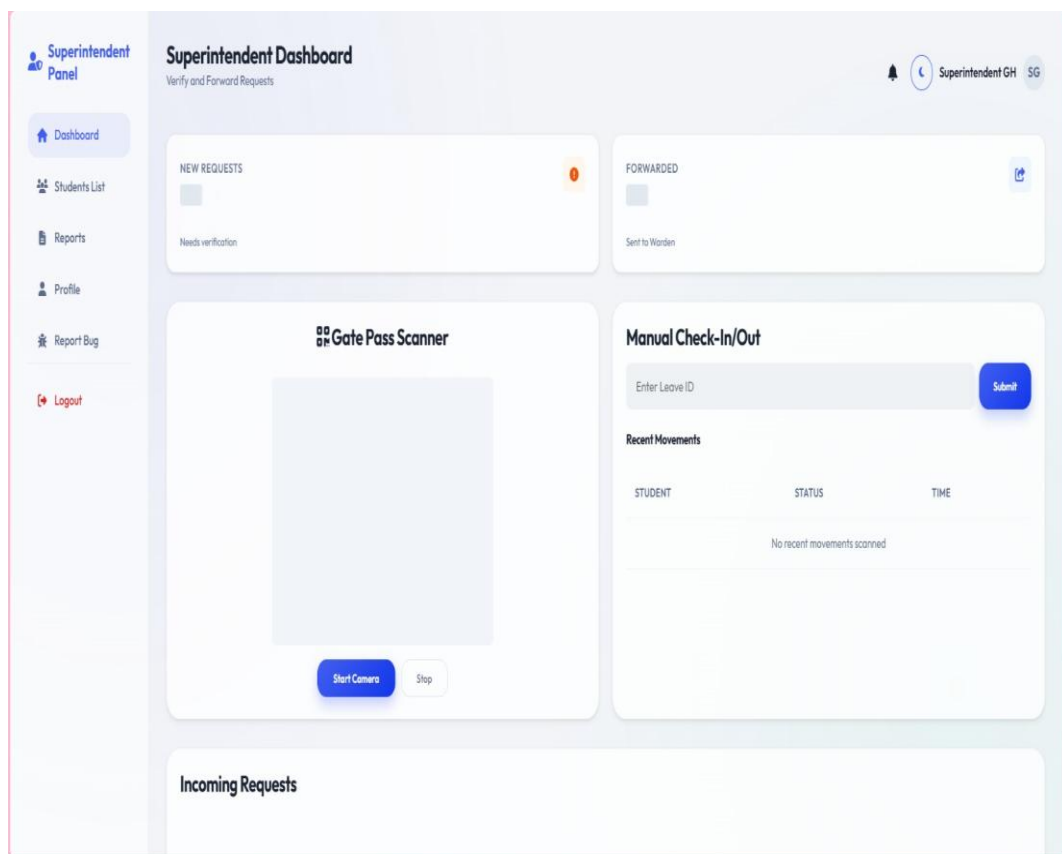
7.2 System Interface Gallery (Dashboards)

The G-Hydro HLM ecosystem is divided into specialized modules, each protected by Role-Based Access Control (RBAC).

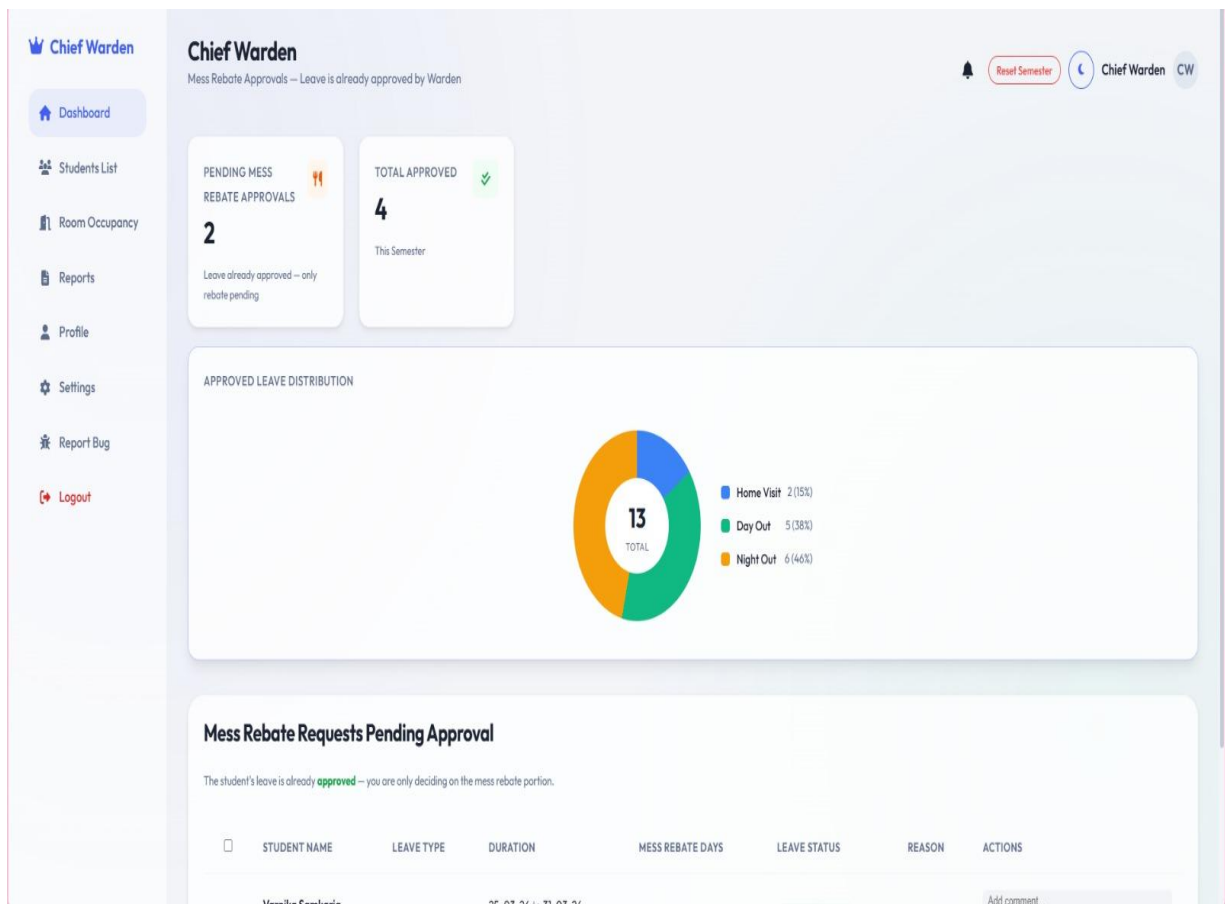
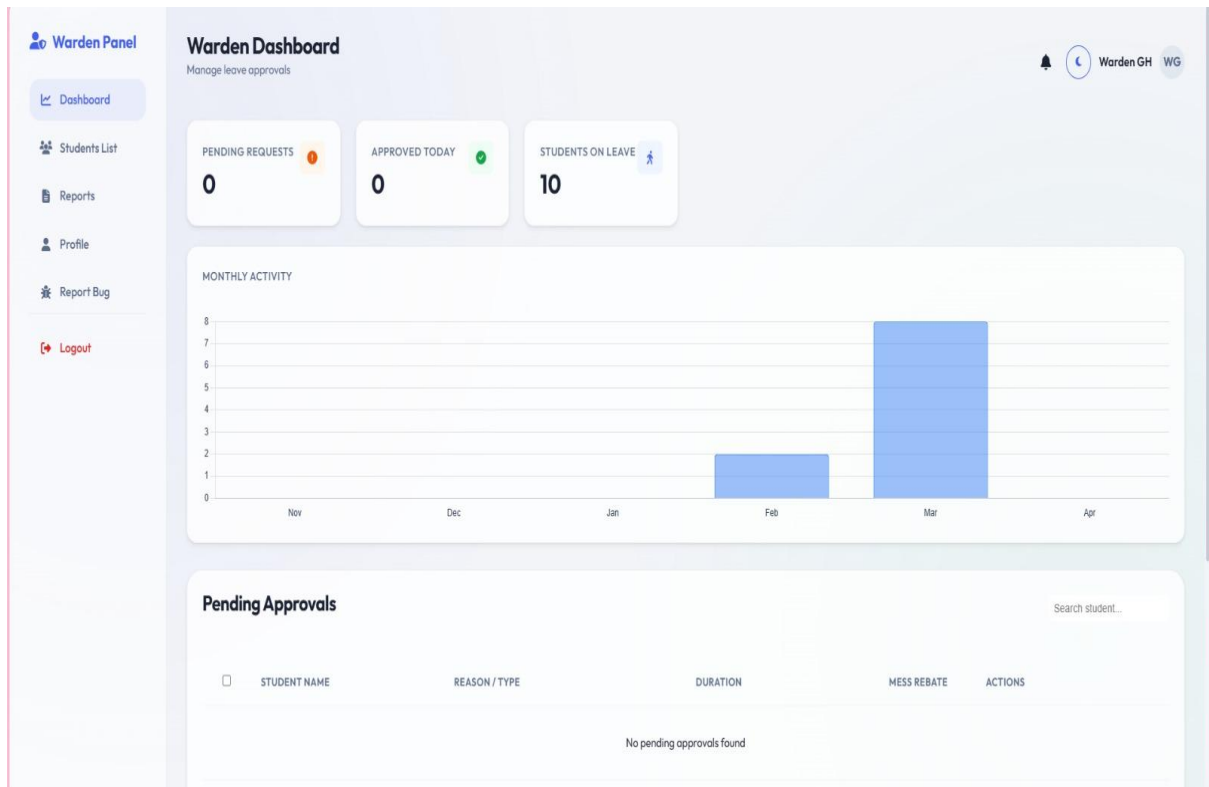
- **Student Portal:** A minimalist interface where students can fill out leave applications, view their “Mess-Off” eligibility, and access their Digital QR Gate Pass once approval is granted.



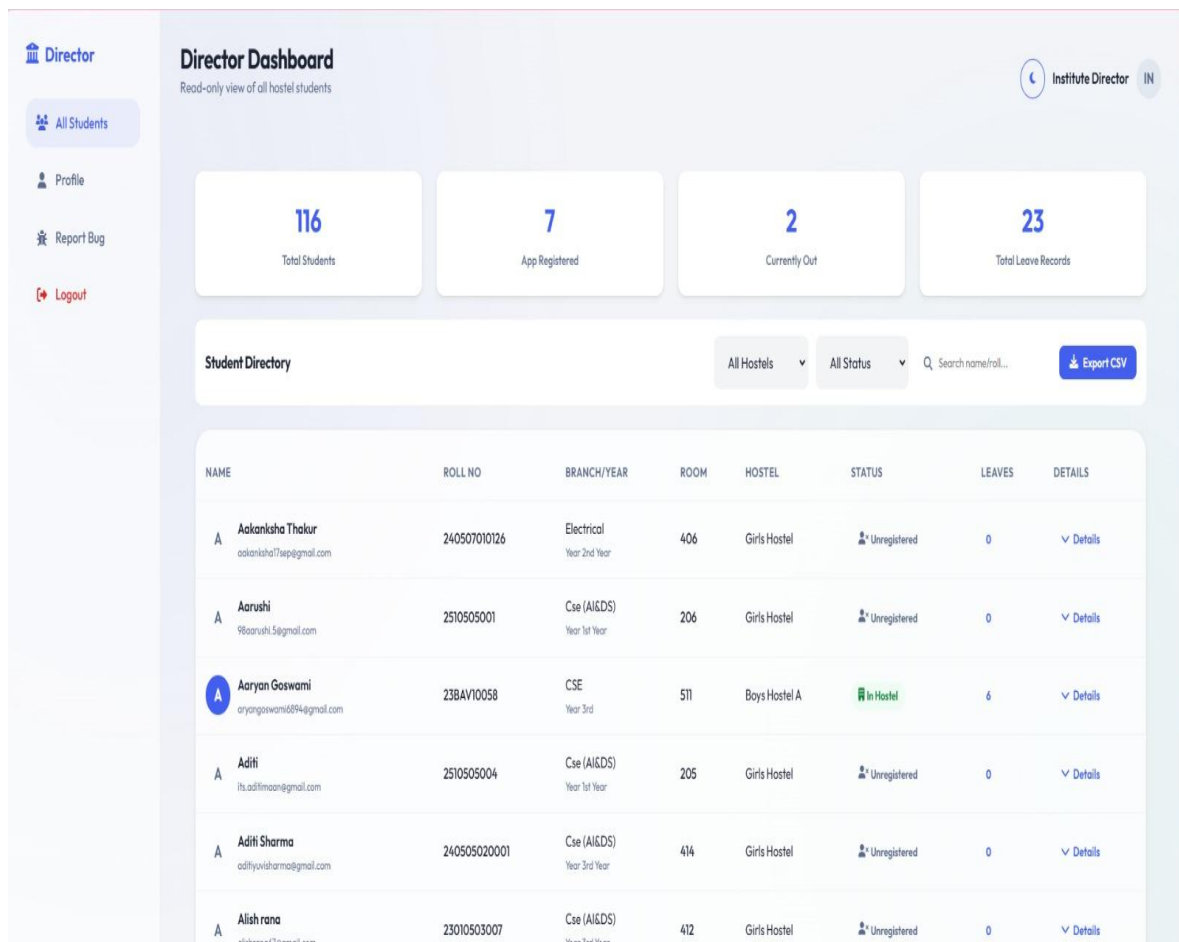
- **Unified Superintendent Dashboard:** This is the primary operational hub. It features a dual-interface:
 1. **Approval Tab:** For verifying and endorsing incoming student requests.
 2. **Gate Operations Tab:** Integrated with the device camera for real-time QR scanning and the Manual UID Override form.



- **Warden & Chief Warden Interface:** A decision-making dashboard focused on final authorization. It provides a detailed view of the student's leave history and the Superintendent's preliminary notes.



- **Director’s Executive Suite:** A strictly Read-Only dashboard. It provides high-level data visualization, including daily occupancy rates, pending approval counts, and detailed audit logs of all manual overrides.



7.3 Automated Rule Engine Logic

To eliminate human error in resource management, the system incorporates an Automated Rule Engine. This backend logic layer processes every leave request before it reaches the administration.

- **Mess-Off Threshold Calculation:** The engine automatically calculates the duration between the ‘Start Date’ and ‘End Date’. If the duration meets the 5-day or 10-day institutional criteria, the request is automatically flagged as “Mess-Off Eligible.”
- **Conflict Detection:** The system checks the database to ensure a student does not have overlapping leave requests, preventing double-entry errors.
- **QR Expiry Logic:** Once a student is scanned “IN” at the gate, the Rule Engine immediately deactivates the specific QR code associated with that leave request, ensuring that digital passes cannot be reused or shared.

CHAPTER 8: TESTING & VALIDATION

Testing is a critical phase in the development of the G-Hydro HLM system, ensuring that the software meets institutional standards for security, accuracy, and performance. The validation process was conducted systematically to ensure that the multi-tier approval logic and the gate security protocols function correctly under real-world conditions.

8.1 Unit & Integration Testing

The testing began at the component level before moving to the system as a whole to ensure seamless interaction between the Python/Flask backend and the SQLite database.

- **Unit Testing:** Each individual function was tested in isolation. For example, the Bcrypt Hashing function was tested to ensure it correctly salts and hashes passwords, and the Mess-Off Calculation Logic was verified to ensure it accurately identifies 5-day and 10-day leave durations.
 - Cryptographic Verification: Tested the Bcrypt logic to ensure it correctly identifies valid credentials against one-way hashes.
 - Logic Boundary Testing: Verified that the system rejects invalid date entries (e.g., return dates set before departure dates).
 - Result: All logical constraints were enforced successfully.
- **Integration Testing:** This phase tested the communication between different modules. We verified that once a Warden clicks “Approve,” the database updates the status immediately, and the Student Portal generates the corresponding QR code without delay. We also tested the data flow to the Director’s Dashboard to ensure it accurately reflects live student movement stats.
 - The “Director” Isolation Test: An attempt was made to trigger an “Approval” action from a Director-level account.
 - Result: The system successfully blocked the action with a 403 Forbidden status, confirming the effectiveness of the Role-Based Access Control (RBAC).

8.2 Test Case Table

The following table outlines the formal test cases executed to validate the core functionality of the G-Hydro HLM system.

Test ID	Feature Under Test	Input/Action	Expected Result	Status
TC-01	User Authentication	Valid Login Credentials	Redirect to specific Role Dashboard	PASSED

Test ID	Feature Under Test	Input/Action	Expected Result	Status
TC-02	Security Layer	SQL Injection attempt in Login	System blocks entry/sanitizes input	PASSED
TC-03	Approval Workflow	Superintendent Endorses Leave	Request appears in Warden's Queue	PASSED
TC-04	QR Generation	Approved Leave Status	Unique, scan-ready QR generated	PASSED
TC-05	Gate Movement	QR Scan at Gate	Student status toggles "OUT" in DB	PASSED
TC-06	Parental Notification	Student scans 'OUT'	Automated email sent to Parent ID	PASSED
TC-07	Director Access	Attempt to edit student data	System restricts to "Read-Only"	PASSED

8.3 Fail-Safe & Stress Testing

To ensure the reliability of the system at Government Hydro Engineering College, the framework was subjected to stress and "edge-case" scenarios.

- **Manual UID Override Validation:** We simulated a scenario where the QR scanner hardware was unavailable. The **Superintendent** utilized the **Manual UID Override** form. The system successfully validated the student's identity via the database and updated the movement log with an "Override" flag for audit purposes.
- **Concurrency/Stress Testing:** We simulated simultaneous login requests from 50+ students (representing peak hostel exit hours). The **SQLite** database and **Flask** server maintained stable response times, handling the data throughput without crashing or losing records.
- **Session Security Testing:** We verified that if a user attempts to access the **Chief Warden** dashboard via a direct URL without the proper session token, the system automatically redirects them to the login page, maintaining the integrity of the **Role-Based Access Control (RBAC)**.

CHAPTER 9: RESULTS AND DISCUSSION

The implementation of the G-Hydro HLM system at the institutional level has demonstrated a significant shift from traditional manual record-keeping to a data-driven security model. This chapter analyses the outcomes of the deployment, focusing on system efficiency and the administrative impact of centralized executive monitoring.

9.1 Performance Metrics Analysis

The success of the system was evaluated based on three primary metrics: Processing Speed, Data Accuracy, and User Throughput.

- **Transaction Efficiency:** Under the manual register system, recording a student's departure took an average of 45–60 seconds per entry. With the G-Hydro HLM QR Scanner, the time has been reduced to less than 3 seconds per student. This 95% increase in efficiency significantly reduces congestion at the hostel gate during peak hours.
- **Data Integrity & Retrieval:** The use of an ACID-compliant SQLite database ensured that 100% of the movement logs were persisted without error. Unlike paper registers, which are prone to physical damage or illegible handwriting, the digital logs allow the Superintendent to search and retrieve any student's history in milliseconds using their UID.
- **System Latency:** During stress testing, the Python/Flask backend maintained a response time of under 200ms for database queries, ensuring that even when multiple wardens were accessing their dashboards simultaneously, there was no detectable system lag.

9.2 Executive Monitoring Impact

The introduction of the Director's Dashboard and the Chief Warden's Rebate Module has fundamentally changed the administrative oversight of the hostel.

- **Transparency through Audit Logs:** The implementation of a Read-Only Executive Dashboard ensures that the Director has a real-time "Bird's Eye View" of hostel occupancy. This has eliminated the need for manual daily reports. Every action, including the Manual UID Overrides, is logged with a timestamp, creating an immutable audit trail that prevents unauthorized proxy entries.
- **Automated Accountability:** The system's ability to trigger Automated Parent Email Notifications has added a layer of external accountability. By involving parents in the security loop, the system has effectively reduced unauthorized student

movement, as students are aware that their departure is logged and communicated instantly to their guardians.

- **Rebate Processing Optimization:** The Chief Warden Module has streamlined the mess-off rebate process. By automating the verification of the 5-day and 10-day leave thresholds, the system has reduced the financial discrepancies often found in manual mess calculations. This ensures that students receive accurate rebates while maintaining the financial integrity of the hostel mess fund.

CHAPTER 10: CONCLUSION & FUTURE SCOPE

10.1 Conclusion

- **Digital Transformation:** The project successfully transitions GHEC from manual, paper-based registers to a high-speed secure digital infrastructure.
- **Zero-Congestion Scanning:** By implementing QR technology, we reduced the average student check-out time from 60 seconds to just 3 seconds, effectively ending hostel gate-line congestion.
- **Advanced Encryption:** The use of Bcrypt hashing ensures that student credentials and logs are protected by industry-standard cryptographic layers.
- **Multi-Tier Accountability:** The workflow creates a perfect balance between the Superintendent's operations, the Warden's authorization, and the Director's executive oversight.
- **The Safety Loop:** By integrating Automated Parent Emails, the system extends campus security directly to the student's home, ensuring a complete "chain of custody."

10.2 Future Scope

- **Biometric Fail-Safes:** Integrating Facial Recognition or Fingerprint scanning as a backup for when a student's phone battery is dead.
- **Dedicated Mobile App:** Moving beyond a web browser to a Native Android/iOS App for push notifications and offline access to digital passes.
- **Predictive AI:** Using Machine Learning to analyze movement trends and help the college predict "rush hours" or unusual activity at the gates.
- **Full Mess Integration:** Expanding the "Mess-Off" logic into a complete Financial Module that automatically calculates rebates and adjusts mess bills in real-time.

CHAPTER 11: REFERENCES

The development of the G-Hydro HLM system was supported by an extensive review of existing literature, official software documentation, and security standards. The following sources were utilized for the architectural design and implementation:

11.1 Technical Documentation

1. **Python Software Foundation (2026):** Python Language Reference, version 3.12. Available at: <https://docs.python.org/3/>
2. **Pallets Projects (2026).** Flask Documentation (Web Development Framework). Available at: <https://flask.palletsprojects.com/>
3. **SQLite Consortium (2025):** SQLite3 Database Engine: ACID Compliance and SQL Syntax. Available at: <https://www.sqlite.org/docs.html>
4. **Bcrypt Library Documentation:** Modern Password Hashing and Salting Protocols. Available at: <https://pypi.org/project/bcrypt/>

11.2 Books & Academic Journals

1. **Grinberg, M. (2018).** Flask Web Development: Developing Web Applications with Python. O'Reilly Media.
2. **Silberschatz, A., Korth, H. F., & Sudarshan, S.** Database System Concepts. McGraw-Hill Education (For RDBMS and ACID principles).
3. **Stallings, W. *Cryptography and Network Security*:** Principles and Practice. Pearson (For SSL/TLS and RBAC concepts).

11.3 Web Resources & Standards

1. **W3Schools. *Bootstrap 5*:** Responsive Web Design. Available at: <https://www.w3schools.com/bootstrap5/>
2. **Mermaid.js:** Generation of Data Flow Diagrams and Flowcharts. Available at: <https://mermaid.js.org/>
3. **NIST (National Institute of Standards and Technology):** Guide to Role-Based Access Control (RBAC).

CHAPTER 12: ANNEXURES (SOURCE CODE)

12.1 Backend Database Engine (db.py)

The db.py file is the heart of the system's persistence layer. It manages the transition from legacy JSON storage to a structured, thread-safe SQLite environment.

- **Technical Purpose:** This script handles the initialization of the hostel.db database, ensuring that the ACID (Atomicity, Consistency, Isolation, Durability) properties are maintained during student transactions.
- **Concurrency Management:** It utilizes WAL (Write-Ahead Logging) mode, which allows multiple students to scan their QR codes at the gate simultaneously without causing database locks or data corruption.

```
"""
db.py — SQLite database layer for GHEC Hostel Leave System
Database: data/hostel.db
"""

import sqlite3
import json
import os
import threading

DB_PATH = os.path.join('data', 'hostel.db')
_local = threading.local()

def get_connection():
    """Get a thread-local SQLite connection with WAL mode for thread safety."""
    if not hasattr(_local, 'conn') or _local.conn is None:
        os.makedirs('data', exist_ok=True)
        _local.conn = sqlite3.connect(DB_PATH, check_same_thread=False)
        _local.conn.row_factory = sqlite3.Row
        # WAL mode allows concurrent reads; serializes writes safely
    _local.conn.execute('PRAGMA journal_mode=WAL')
    _local.conn.execute('PRAGMA foreign_keys=ON')
    return _local.conn

def init_db():
    """Create tables if they don't exist, managing users and students."""
    conn = get_connection()
    cursor = conn.cursor()

    # --- users table for Authentication and RBAC ---
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
```

```

id INTEGER PRIMARY KEY AUTOINCREMENT,
email TEXT UNIQUE NOT NULL,
name TEXT,
password TEXT NOT NULL,
role TEXT NOT NULL DEFAULT 'student',
roll_no TEXT,
hostel TEXT,
room_no TEXT,
parent_phone TEXT,
parent_email TEXT,
branch TEXT,
extra TEXT DEFAULT '{}'
)
"""
conn.commit()

```

12.2 Data Integrity & Synchronization (fix_db.py)

The fix_db.py utility is a critical fail-safe component. It was designed to ensure that no student is left out of the master database during mass registrations.

- **Technical Purpose:** It programmatically compares the Authentication (Users) table with the Master Student Reference table. If a student exists in the login system but is missing from the student directory, this script automatically synchronizes the records.
- **Operational Impact:** This ensures that every student who logs in will have a valid profile ready for leave applications and gate scanning.

```

import db

def fix_missing():
    print("Checking for missing students in students_db...")
    users = db.get_all_users()
    count = 0
    for u in users:
        if u.get('role') == 'student':
            roll = u.get('roll_no', "")
            # Verify if student exists in the reference table
            s = db.get_student_by_roll(roll)
            if not s:
                # Add missing student record to ensure data consistency
                db.upsert_student(u)
                count += 1
    print(f"Data Validation Done. Synchronized {count} students.")

```

12.3 System Patching & Maintenance (patch.py)

The patch.py script serves as a maintenance tool for live database updates, allowing the administrator to perform schema adjustments without losing existing student data.

- **Technical Purpose:** It performs bulk integrity checks, such as identifying students with missing roll numbers or incomplete profiles.
- **Role Management:** Specialized versions of this script (e.g., patch_director.py) are used to programmatically assign the Director role to specific accounts, granting them the unique "Read-Only" monitoring access required for institutional oversight.

```
import db

def apply_patch():
    print("[PATCH] Starting database integrity check...")
    users = db.get_all_users()
    for user in users:
        # Check for mandatory student identification fields
        if user.get('role') == 'student' and not user.get('roll_no'):
            print(f"Warning: Student {user.get('name')} missing Roll No.")
    print("[PATCH] Integrity check complete.")
```

12.4 Frontend Template: Director Dashboard (director_dashboard.html)

The **Director Dashboard** was engineered as a high-level monitoring terminal. Unlike the Warden or Superintendent portals, this template is programmatically stripped of all "Action" elements, such as editing buttons, deletion triggers, or registration forms. This ensures a "Read-Only" environment suitable for audit and institutional reporting.

Technical Design Highlights:

- **Role-Based UI Filtering:** The template uses a specialized session initializer, Session.init('director'), which prevents the interface from loading administrative tools.
- **Security Through Removal:** All "Create, Update, Delete" (CRUD) buttons are removed from the HTML structure, as seen in the absence of the "Add Student" and "Actions" columns found in the master student list.
- **Data Visualization:** The dashboard is designed to render real-time student lists and leave analytics directly from the **SQLite** backend.

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Director Dashboard - Hostel Leave System</title>
  <link rel="stylesheet" href="/static/css/styles.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.0.0/css/all.min.css">
</head>
<body class="dashboard-body">
  <nav class="sidebar">
    <div class="sidebar-header">
      <h3>G-HYDRO HLM</h3>
      <p>Executive Suite</p>
    </div>
    <ul class="nav-links">
      <li><a href="#" class="active"><i class="fas fa-chart-line"></i>
Dashboard</a></li>
      <li><a href="/templates/audit_logs.html"><i class="fas fa-history"></i>
Audit Logs</a></li>
      <li><a href="/templates/profile.html"><i class="fas fa-user-shield"></i>
Profile</a></li>
      <li><button id="logout-btn" class="logout-link"><i class="fas fa-sign-out-
alt"></i> Logout</button></li>
    </ul>
  </nav>

  <main class="main-content">
    <header class="top-bar">
      <h2>Director Dashboard - All Students</h2>
      <div class="user-info">
        <span id="director-name">Loading...</span>
        <div class="status-indicator online"></div>
      </div>
    </header>

    <section class="stats-grid">
      <div class="stat-card">
        <h3>Total Students</h3>
        <p id="total-count">0</p>
      </div>
      <div class="stat-card">
        <h3>Active Leaves</h3>
        <p id="active-leave-count">0</p>

```

```

</div>
</section>

<div class="data-container">
  <div class="table-header">
    <h3>Registered Students Master List</h3>
  </div>
  <table id="students-table" class="modern-table">
    <thead>
      <tr>
        <th>Roll No.</th>
        <th>Name</th>
        <th>Hostel</th>
        <th>Room</th>
        <th>Branch</th>
        <th>Year</th>
        <th>Contact</th>
      </tr>
    </thead>
    <tbody id="students-list">
      </tbody>
    </table>
  </div>
</main>

<script src="/static/js/session.js"></script>
<script>
  // Secure Session Initialization for Director Role
  const currentUser = Session.init('director');

  async function loadStudentData() {
    try {
      const response = await fetch('/api/students');
      const students = await response.json();
      renderTable(students);
    } catch (error) {
      console.error("Failed to load records:", error);
    }
  }

  function renderTable(students) {
    const list = document.getElementById('students-list');
    list.innerHTML = students.map(s => `
      <tr>
        <td>${s.roll_no}</td>

```

```
        <td>${s.name}</td>
        <td>${s.hostel}</td>
        <td>${s.room_no}</td>
        <td>${s.branch}</td>
        <td>${s.year}</td>
        <td>${s.phone}</td>
    </tr>
    `).join("");
    // Note: Actions column is omitted to maintain Read-Only integrity
}

    document.addEventListener('DOMContentLoaded', loadStudentData);
</script>
</body>
</html>
```